

FOOD DELIVERY ORDER DECISION SYSTEM

Practical Record

AIM

To write a Python program that automates the order decision process of a food delivery platform such as Swiggy or Zomato, using only conditional statements.

PROBLEM STATEMENT

A food delivery platform wants to automate its order decision process. The program accepts order amount, delivery distance, customer type, customer and restaurant ratings, preparation time, payment method, weather condition, demand level, peak-hour status and previous cancellations. Using only conditional statements such as if, elif, else, nested conditions, and and or, it decides whether the order should be accepted, rejected or sent for manual review, along with delivery charges, discounts, priority status, cancellation risk, restaurant status and the final payable amount. Only core Python is used, without NumPy, Pandas, datasets or any external library.

ALGORITHM

1. Start.
2. Read all eleven inputs from the user.
3. Decide the restaurant status from its rating and preparation time.
4. Decide the cancellation risk from previous cancellations, payment method and customer rating.
5. Using nested if / elif / else, decide the order status as Accepted, Rejected or Manual Review.
6. Set the manual review flag if the order needs approval.
7. Calculate the delivery charge from distance, then adjust it for weather, peak hour, demand and customer type.
8. Calculate the discount percentage from customer type and order amount, then adjust it for payment method, ratings and demand.
9. Decide the priority of delivery.
10. Place the order in a final category.
11. Calculate the final payable amount as amount - discount + delivery charge + COD fee.
12. Print the final order report.
13. Stop.

SOURCE CODE

```
# Food Delivery Order Decision System

print("=" * 50)
print("      FOOD DELIVERY ORDER DECISION SYSTEM")
print("=" * 50)

# take all the inputs first
amount = float(input("Order amount (Rs)                : "))
distance = float(input("Delivery distance (km)          : "))
customer = input("Customer type (new/regular/premium) : ").strip().lower()
cust_rating = float(input("Customer rating (1-5)        : "))
rest_rating = float(input("Restaurant rating (1-5)      : "))
prep_time = int(input("Preparation time (minutes)       : "))
payment = input("Payment method (upi/cod/card)         : ").strip().lower()
weather = input("Weather (clear/rain/storm)            : ").strip().lower()
demand = input("Demand level (low/normal/high)        : ").strip().lower()
peak = input("Peak hour (yes/no)                      : ").strip().lower()
cancellations = int(input("Previous cancellations      : "))

# check if the restaurant is good enough to take the order
if rest_rating >= 4.0 and prep_time <= 30:
    restaurant_status = "Active - Reliable"
elif rest_rating >= 3.0 and prep_time <= 45:
    restaurant_status = "Active - Normal"
elif rest_rating >= 2.5 or prep_time <= 60:
    restaurant_status = "Active - Under Watch"
else:
    restaurant_status = "Temporarily Suspended"

# how likely is this customer to cancel
if cancellations >= 5 or (cancellations >= 3 and payment == "cod"):
    cancel_risk = "High"
elif cancellations >= 3 or (cancellations >= 1 and cust_rating < 3.0):
    cancel_risk = "Medium"
elif payment == "cod" and amount > 1000:
    cancel_risk = "Medium"
else:
    cancel_risk = "Low"

# main decision - accept, reject or send for review
if restaurant_status == "Temporarily Suspended":
    order_status = "Rejected"
    reason = "Restaurant suspended (low rating / very slow prep)"
elif amount < 100:
    order_status = "Rejected"
    reason = "Below minimum order value of Rs 100"
elif distance > 20:
    order_status = "Rejected"
    reason = "Outside 20 km delivery range"
elif cancel_risk == "High":
    # money already paid, so worth a second look instead of rejecting
    if payment == "upi" or payment == "card":
        order_status = "Manual Review"
        reason = "High cancellation risk, but payment is UPI/Card"
    else:
        order_status = "Rejected"
        reason = "High cancellation risk on COD order"
elif weather == "storm":
    if distance <= 5 and amount >= 300:
        order_status = "Manual Review"
        reason = "Storm, but short distance and good value"
    else:
        order_status = "Rejected"
        reason = "Storm - delivery unsafe at this distance"
```

```

elif distance > 15 and payment == "cod":
    order_status = "Manual Review"
    reason = "Long distance COD order"
elif cancel_risk == "Medium" and amount > 1500:
    order_status = "Manual Review"
    reason = "High value order from medium risk customer"
elif cust_rating < 2.5:
    order_status = "Manual Review"
    reason = "Customer rating below 2.5"
elif restaurant_status == "Active - Under Watch" and amount > 1000:
    order_status = "Manual Review"
    reason = "High value order from a restaurant under watch"
else:
    order_status = "Accepted"
    reason = "All checks passed"

if order_status == "Manual Review":
    manual_review = "YES - " + reason
else:
    manual_review = "Not required"

# delivery charge, base amount depends on distance
if distance <= 3:
    delivery_charge = 20
elif distance <= 7:
    delivery_charge = 40
elif distance <= 12:
    delivery_charge = 60
else:
    delivery_charge = 90

# bad weather costs extra
if weather == "rain":
    delivery_charge = delivery_charge + 15
elif weather == "storm":
    delivery_charge = delivery_charge + 30

if peak == "yes":
    delivery_charge = delivery_charge + 20

if demand == "high":
    delivery_charge = delivery_charge + 25
elif demand == "low":
    delivery_charge = delivery_charge - 10

# premium customers get free or cheaper delivery
if customer == "premium" and amount >= 500:
    delivery_charge = 0
elif customer == "premium" or amount >= 800:
    delivery_charge = delivery_charge - 20

if delivery_charge < 0:
    delivery_charge = 0

# discount depends on customer type and how big the order is
if customer == "premium":
    if amount >= 1000:
        discount_pct = 20
    elif amount >= 500:
        discount_pct = 15
    else:
        discount_pct = 10
elif customer == "regular":
    if amount >= 1000:
        discount_pct = 12
    elif amount >= 500:
        discount_pct = 8
    else:

```

```

        discount_pct = 5
    else:
        # new customers get the joining offer
        if amount >= 500:
            discount_pct = 25
        else:
            discount_pct = 15

    if payment == "upi" or payment == "card":
        discount_pct = discount_pct + 5

    if rest_rating >= 4.5 and cust_rating >= 4.5:
        discount_pct = discount_pct + 3

    # no big discounts when everyone is ordering
    if demand == "high" or peak == "yes":
        discount_pct = discount_pct - 5

    if discount_pct > 30:
        discount_pct = 30
    elif discount_pct < 0:
        discount_pct = 0

    discount = amount * discount_pct / 100

    # who gets their food first
    if customer == "premium" and cust_rating >= 4.0:
        priority = "Priority Delivery"
    elif amount >= 1500 or (customer == "regular" and cust_rating >= 4.5 and amount >= 800):
        priority = "Priority Delivery"
    elif prep_time <= 15 and distance <= 5:
        priority = "Express Delivery"
    elif weather == "storm" or distance > 15 or prep_time > 45:
        priority = "Delayed Delivery"
    else:
        priority = "Standard Delivery"

    if order_status == "Rejected":
        priority = "Not Applicable"

    # put the order in a final bucket
    if order_status == "Rejected":
        category = "Cancelled Order"
    elif order_status == "Manual Review":
        category = "Pending Approval"
    elif amount >= 1500 and customer == "premium":
        category = "Elite Order"
    elif amount >= 1000 or priority == "Priority Delivery":
        category = "Premium Order"
    elif amount >= 400:
        category = "Standard Order"
    else:
        category = "Basic Order"

    # final bill
    cod_fee = 0
    if order_status == "Rejected":
        # nothing is charged if we did not take the order
        final_amount = 0.0
        discount = 0.0
        discount_pct = 0
        delivery_charge = 0
    else:
        if payment == "cod":
            cod_fee = 20
        final_amount = amount - discount + delivery_charge + cod_fee
        if final_amount < 0:

```

```

        final_amount = 0.0

# print everything out
print()
print("=" * 50)
print("                FINAL ORDER REPORT")
print("=" * 50)
print("Order Status      :", order_status)
print("Reason            :", reason)
print("Restaurant Status   :", restaurant_status)
print("Cancellation Risk    :", cancel_risk)
print("Manual Review        :", manual_review)
print("Priority Delivery     :", priority)
print("Final Category       :", category)
print("-" * 50)
print("Order Amount         : Rs", round(amount, 2))
print("Discount Percent      :", discount_pct, "%")
print("Discount Amount       : Rs", round(discount, 2))
print("Delivery Charge      : Rs", round(delivery_charge, 2))
if cod_fee > 0:
    print("COD Handling Fee      : Rs", cod_fee)
print("-" * 50)
print("FINAL PAYABLE        : Rs", round(final_amount, 2))
print("=" * 50)

```

OUTPUT

Sample Run 1 : Order Accepted

```
=====
FOOD DELIVERY ORDER DECISION SYSTEM
=====
Order amount (Rs)           : 1200
Delivery distance (km)      : 4
Customer type (new/regular/premium) : premium
Customer rating (1-5)       : 4.6
Restaurant rating (1-5)     : 4.7
Preparation time (minutes)  : 20
Payment method (upi/cod/card) : upi
Weather (clear/rain/storm)  : clear
Demand level (low/normal/high) : normal
Peak hour (yes/no)          : no
Previous cancellations      : 0

=====
FINAL ORDER REPORT
=====
Order Status      : Accepted
Reason            : All checks passed
Restaurant Status : Active - Reliable
Cancellation Risk  : Low
Manual Review     : Not required
Priority Delivery  : Priority Delivery
Final Category    : Premium Order
-----
Order Amount      : Rs 1200.0
Discount Percent  : 28 %
Discount Amount   : Rs 336.0
Delivery Charge   : Rs 0
-----
FINAL PAYABLE     : Rs 864.0
=====
```

Sample Run 2 : Order Rejected

```
=====
FOOD DELIVERY ORDER DECISION SYSTEM
=====
Order amount (Rs)           : 500
Delivery distance (km)      : 12
Customer type (new/regular/premium) : regular
Customer rating (1-5)       : 4.0
Restaurant rating (1-5)     : 4.0
Preparation time (minutes)  : 25
Payment method (upi/cod/card) : cod
Weather (clear/rain/storm)  : storm
Demand level (low/normal/high) : high
Peak hour (yes/no)          : yes
Previous cancellations      : 0

=====
FINAL ORDER REPORT
=====
Order Status      : Rejected
Reason            : Storm - delivery unsafe at this distance
Restaurant Status : Active - Reliable
Cancellation Risk  : Low
Manual Review     : Not required
Priority Delivery  : Not Applicable
Final Category    : Cancelled Order
-----
Order Amount      : Rs 500.0
Discount Percent  : 0 %
Discount Amount   : Rs 0.0
Delivery Charge   : Rs 0
-----
```

```
-----  
FINAL PAYABLE      : Rs 0.0  
=====
```

Sample Run 3 : Sent for Manual Review

```
=====
```

FOOD DELIVERY ORDER DECISION SYSTEM

```
=====
```

Order amount (Rs)	: 800
Delivery distance (km)	: 6
Customer type (new/regular/premium)	: new
Customer rating (1-5)	: 3.5
Restaurant rating (1-5)	: 4.2
Preparation time (minutes)	: 25
Payment method (upi/cod/card)	: upi
Weather (clear/rain/storm)	: clear
Demand level (low/normal/high)	: normal
Peak hour (yes/no)	: no
Previous cancellations	: 6

```
=====
```

FINAL ORDER REPORT

```
=====
```

Order Status	: Manual Review
Reason	: High cancellation risk, but payment is UPI/Card
Restaurant Status	: Active - Reliable
Cancellation Risk	: High
Manual Review	: YES - High cancellation risk, but payment is UPI/Card
Priority Delivery	: Standard Delivery
Final Category	: Pending Approval

```
-----
```

Order Amount	: Rs 800.0
Discount Percent	: 30 %
Discount Amount	: Rs 240.0
Delivery Charge	: Rs 20

```
-----
```

FINAL PAYABLE	: Rs 580.0
---------------	------------

```
=====
```

RESULT

The program was executed successfully. The order status, delivery charge, discount, priority status, cancellation risk, restaurant status, manual review status, final category and final payable amount change correctly for different sets of input values, as shown in the three sample runs above.